Simulation Final

Justin Schretzman Medich

I. Project Overview and Algorithms

A. Project Goals

This project implements a physically-based cloth simulation system with robust collision handling. The primary goals were to:

- Create a realistic cloth simulation using position-based dynamics
- Implement efficient collision detection for self-collisions and environment interactions
- Model realistic physical behaviors including friction
- Prevent cloth self-penetration while maintaining visual quality

The implementation builds upon the techniques described in Bridson et al.'s paper "Robust Treatment of Collisions, Contact and Friction for Cloth Animation," adapting their approach to a position-based dynamics framework.

B. Key Algorithms

1) Position-Based Dynamics: The simulation uses a position-based dynamics (PBD) approach where the cloth is represented as a triangle mesh with point masses at vertices. Unlike force-based methods, PBD directly manipulates positions to satisfy constraints. This offers several advantages:

- Unconditionally stable simulation
- Intuitive control over physical behaviors
- Direct handling of collisions as position constraints

The basic simulation loop follows these steps:

1) Predict new vertex positions using explicit Euler integration
2) Detect and resolve collisions
3) Apply internal constraint projections (using pre-existing constraint solvers)
4) Update velocities based on position changes

2) Collision Detection: Efficient collision detection is crucial for cloth simulation, as every point on the cloth surface potentially interacts with all other points. Our implementation uses a bounding volume hierarchy (BVH) to accelerate collision queries:

1) **BVH Construction**: A bottom-up approach where leaf nodes represent triangles, and interior nodes group adjacent nodes. The BVH is rebuilt each time step to account for the cloth's movement.
2) **Proximity Detection**: Detects pairs of cloth elements (vertex-triangle or edge-edge) that are within the cloth thickness distance. This prevents penetrations before they occur.
3) **Continuous Collision Detection**: Detects collisions that might occur between discrete time steps by solving for the time of impact between moving elements:
   - For edge-edge collisions, we solve a cubic equation to find when the distance between edges equals the cloth thickness
   - For vertex-triangle collisions, we compute when a moving vertex crosses the plane of a triangle

3) Collision Response: Once collisions are detected, we apply appropriate responses:

1) **Repulsion Forces**: When cloth elements are in close proximity but not yet colliding, we apply short-range repulsion forces to prevent penetration. These forces are proportional to the proximity distance and are applied in the direction of the surface normal.
2) **Collision Resolution**: For actual collisions, we apply impulses to the colliding elements:
   - For edge-edge collisions, impulses are distributed to the four vertices based on barycentric coordinates
   - For vertex-triangle collisions, impulses are applied to the vertex and distributed across the triangle vertices using barycentric weights
3) **Friction Modeling**: We implement Coulomb friction for both static and kinetic friction:
   - Static friction: When relative tangential velocity is small, we prevent sliding
   - Kinetic friction: When sliding occurs, we apply a friction impulse proportional to the normal force in the direction opposing motion

4) External Collisions: The simulation handles two types of environmental collisions:

1) **Floor Collision**: Simple plane-point collision detection with offset by cloth thickness
2) **Sphere Collision**: Distance-based detection between vertices and the sphere's surface

Both collision types are resolved using projection and friction impulses similar to self-collisions.

C. Mathematical Foundations

1) Vertex-Triangle Collision: For a vertex with position $\mathbf{p}$ and velocity $\mathbf{v}$ approaching a triangle with vertices $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$ and velocities $\mathbf{v_a}$, $\mathbf{v_b}$, $\mathbf{v_c}$, we calculate:

1) Triangle normal at start: $\mathbf{n_0} = normalize((\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a}))$
2) Signed distance: $d_0 = (\mathbf{p} - \mathbf{a}) \cdot \mathbf{n_0}$
3) Relative velocity: $v_n = \mathbf{v} \cdot \mathbf{n_0} - (\mathbf{v_a} + \mathbf{v_b} + \mathbf{v_c})/3 \cdot \mathbf{n_0}$

If $d_0 > 0$ and $v_n < 0$, a collision occurs at time $t = (cloth_thickness - d_0)/v_n$.

At collision time, we calculate barycentric coordinates (u, v, w) of the projection of $\mathbf{p}$ onto the triangle plane, and apply impulses if the projection lies within the triangle $(u, v, w \geq 0)$.

2) Edge-Edge Collision: For edges $(\mathbf{p_1}, \mathbf{p_2})$ and $(\mathbf{p_3}, \mathbf{p_4})$ with corresponding velocities, we:

1) Calculate edge vectors $\mathbf{e_1} = \mathbf{p_2} - \mathbf{p_1}$, $\mathbf{e_2} = \mathbf{p_4} - \mathbf{p_3}$
2) Find closest points on both edges using parametric coordinates a and b
3) Compute collision time by solving when distance equals cloth thickness
4) Apply impulses to all four vertices weighted by a and b

The collision normal is $\mathbf{n} = normalize(\mathbf{e_1} \times \mathbf{e_2})$, and impulses are applied along this direction.

3) Proximity Repulsion: To handle proximity, we detect when elements are closer than the cloth thickness but not yet colliding. For such pairs, we apply repulsion forces of magnitude:

$$F_{repulsion} = k \cdot d \cdot \mathbf{n} \tag{1}$$

Where k is a stiffness constant, d is the depth of proximity (cloth thickness minus actual distance), and $\mathbf{n}$ is the contact normal.

4) Friction Model: For friction, we use the Coulomb model where the maximum friction force is proportional to the normal force:

$$F_{friction} \leq \mu \cdot F_{normal} \tag{2}$$

We distinguish between static and kinetic friction:

- If the relative tangential velocity v_T is below a threshold, we apply a constraint to maintain zero relative velocity (static friction)
- Otherwise, we apply a friction impulse of magnitude $\mu \cdot F_{normal}$ in the direction opposite to v_T (kinetic friction)

This dual approach creates realistic frictional behavior where objects initially resist movement but slide when sufficient force is applied.

II. Implementation Details and Limitations

A. Code Architecture

The collision detection and response system is implemented within a position-based dynamics framework. The implementation is structured around these primary components:

1) **CollisionDetector**: A class that handles BVH construction and collision/proximity detection
2) **CollisionCandidate**: A struct representing potential collisions of different types (edge-edge, vertex-triangle)
3) **BVHNode** and **AABB**: Supporting structures for the bounding volume hierarchy
4) **SimParameters**: Holds all tunable simulation parameters

The main simulation loop in `simulateOneStep()` orchestrates the entire process:

1) Predict new positions using the pre-existing dynamics system
2) Construct BVH and detect collisions
3) Resolve collisions using iterative impulses
4) Update velocities based on collision resolution
5) Apply external constraints (pinning, floor, sphere)

Data is stored in Eigen matrices for efficient numerical operations:

- `Q`: Matrix of vertex positions
- `Qdot`: Matrix of vertex velocities
- `F`: Matrix of triangle indices

B. Collision Detection Implementation

The BVH-based collision detection system provides an efficient spatial acceleration structure:

1) BVH Construction: The BVH is built bottom-up in the `buildBVH()` method:

1) Create leaf nodes for each triangle with axis-aligned bounding boxes (AABBs)
2) Expand bounding boxes by the cloth thickness to account for proximity detection
3) Group adjacent nodes in pairs to form parent nodes
4) Continue merging until a single root node is formed

This approach creates a binary tree with $O(n)$ nodes for n triangles, enabling $O(\log n)$ traversal time for collision queries.

2) Collision Detection Pipeline: Collision detection operates in these key phases:

1) `detectCollisions()`: For each triangle, check potential collisions against the BVH
2) `checkCollisionsAgainstBVH()`: Recursively traverses the BVH to find overlapping triangles
3) Individual collision tests: `edgeEdgeCollision()` and `vertexTriangleCollision()`

Neighboring triangles are excluded from collision checks to avoid false positives. This is implemented by checking if triangles share any vertices before performing detailed collision tests.

3) Proximity Detection: The `checkProximity()` method uses the same BVH structure but with a different test criteria:

- Instead of looking for time of impact, it measures current distance
- Reports when elements are closer than the cloth thickness
- Used for applying repulsion forces to prevent future collisions

This two-stage approach (proximity + collision) is essential for robust cloth simulation, as it prevents penetrations before they occur.

C. Collision Response Implementation

1) Impulse-Based Resolution: Collision response is handled through velocity updates in the `resolveCollision()` method:

1) For vertex-triangle collisions:

- Compute relative velocity along normal direction
- Calculate impulse magnitude based on relative normal velocity
- Distribute impulse to vertex and triangle vertices using barycentric weights

2) For edge-edge collisions:
   - Calculate impulse based on relative velocity along edge cross-product normal
   - Distribute impulse to all four vertices using edge weights

The implemented model uses a zero coefficient of restitution to model inelastic collisions, which is appropriate for cloth.

2) Friction Implementation: Friction is implemented in the `applyFrictionImpulse()` method using the Coulomb friction model:

1) Calculate normal impulse magnitude from collision response
2) Compute maximum friction impulse as $\mu \times normalImpulse$
3) Calculate actual friction impulse based on relative tangential velocity
4) Apply friction impulses to vertices in the direction opposite to tangential velocity

Both static and kinetic friction modes are supported, transitioning automatically based on relative tangential velocity.

3) External Collisions: External collisions with the floor and sphere are handled in `resolveExternalCollisions()`:

1) Floor collisions: Project vertices above the floor plane and apply friction to horizontal movement
2) Sphere collisions: Project vertices outside the sphere and apply friction along the tangential direction

These external collisions use the same friction model as cloth self-collisions, providing consistent behavior.

D. Optimization Techniques

Several optimization strategies were employed to improve performance:

1) **Multi-phase collision handling**: Using repulsion forces for general proximity and exact impulses only for actual collisions
2) **Iterative collision resolution**: Limited to a configurable number of iterations (default: 10) to bound computation time
3) **Spatial pruning**: Using BVH to reduce collision pairs from $O(n^2)$ to $O(n \log n)$ in the average case
4) **Early rejection**: Quick AABB tests before computing detailed collision responses

E. Known Limitations

Despite our best efforts, the implementation has several limitations:

1) **Performance bottlenecks**:
   - BVH rebuilding each frame is costly for high-resolution meshes
   - Collision detection remains the primary performance bottleneck
2) **Robustness issues**:
   - Tunneling can still occur with very high velocities due to discrete time stepping (Issue when dropping cloth from high height onto floor)
   - Complex collision scenarios can lead to unstable behavior with conflicting impulses
3) **Friction limitations**:
   - Simple Coulomb model doesn't capture all real-world friction behaviors
   - Static-to-kinetic friction transition can produce visual artifacts
4) **Collision response limitations**:
   - Purely repulsive model lacks accurate compression behavior
   - Multiple simultaneous collisions can lead to "jittering" artifacts
   - Highly constrained configurations (e.g., cloth wedged between objects) can cause instability

F. Future Improvements

Several enhancements could address the current limitations:

1) **Performance improvements**:
   - Incremental BVH updates rather than full rebuilds
   - Spatial hashing for coarse collision culling
   - Parallel collision detection on GPU
2) **Robustness improvements**:
   - Continuous collision detection for all collision types
   - Rigid impact zones for complex collision scenarios
   - Global intersection analysis to resolve multiple conflicting collisions
   - Allowing a variety of different objects to be added to the scene
3) **Accuracy improvements**:
   - More sophisticated friction models including anisotropic friction
   - Air drag and wind simulation
   - Adaptive subdivision in high-curvature regions